```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("/content/AB_NYC_2019[1].csv")
```

```
df.head()
```

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	latitude	longitude	room_type	price	minimum_
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington	40.64749	-73.97237	Private room	149.0	
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	40.75362	-73.98377	Entire home/apt	225.0	
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem	40.80902	-73.94190	Private room	150.0	
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	40.68514	-73.95976	Entire home/apt	89.0	
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	40.79851	-73.94399	Entire home/apt	80.0	

```
df.tail()
```

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	latitude	longitude	room_type	price	m
22054	17744351	1 Bd furnished Bayridge Brooklyn NY	119029523	Ebada	Brooklyn	Fort Hamilton	40.62241	-74.02863	Entire home/apt	115.0	
22055	17746222	Perfect bedroom. Near Subways Columbia CityCol...	16721721	Federico	Manhattan	Harlem	40.81530	-73.95080	Private room	65.0	
22056	17748284	Guest BedRm in quaint NYC neighborhood. NEAR T...	25196982	Nikolas	Queens	Astoria	40.76924	-73.91702	Private room	65.0	
22057	17749273	1 br Woodside - close to LGA!	121096858	Caressa	Queens	Woodside	40.74176	-73.90631	Private room	35.0	
22058	17750238	Great room in a spacious Inwood Apartment	13860679	Max	Manhattan	Inwood	40.86348	-73.00000	NaN	NaN	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22059 entries, 0 to 22058
Data columns (total 16 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   id                              22059 non-null  int64
 1   name                            22044 non-null  object
 2   host_id                         22059 non-null  int64
 3   host_name                       22045 non-null  object
 4   neighbourhood_group             22059 non-null  object
 5   neighbourhood                   22059 non-null  object
 6   latitude                        22059 non-null  float64
```

```
 7   longitude                       22059 non-null  float64
 8   room_type                       22058 non-null  object
 9   price                           22058 non-null  float64
 10  minimum_nights                  22058 non-null  float64
 11  number_of_reviews               22058 non-null  float64
 12  last_review                     18358 non-null  object
 13  reviews_per_month               18358 non-null  float64
 14  calculated_host_listings_count  22058 non-null  float64
 15  availability_365                22058 non-null  float64
dtypes: float64(8), int64(2), object(6)
memory usage: 2.7+ MB
```

```
df.describe()
```

	id	host_id	latitude	longitude	price	minimum_nights	number_of_reviews	reviews_per_month
count	2.205900e+04	2.205900e+04	22059.000000	22059.000000	22058.000000	22058.000000	22058.000000	18358.000000
mean	8.511062e+06	2.447159e+07	40.729520	-73.956852	151.245852	7.408151	35.785248	0.944976
std	5.258526e+06	2.707545e+07	0.053118	0.039701	235.534537	23.883845	58.826793	1.297014
min	2.539000e+03	2.571000e+03	40.499790	-74.242850	10.000000	1.000000	0.000000	0.010000
25%	3.996390e+06	3.977023e+06	40.689765	-73.983265	73.000000	2.000000	1.000000	0.100000
50%	8.520836e+06	1.428111e+07	40.723370	-73.958010	110.000000	3.000000	9.000000	0.360000
75%	1.324998e+07	3.657868e+07	40.763950	-73.941490	175.000000	5.000000	45.000000	1.310000
max	1.775024e+07	1.210969e+08	40.908040	-73.000000	10000.000000	1250.000000	629.000000	16.220000

```
df.isnull().sum()
```

	0
id	0
name	15
host_id	0
host_name	14
neighbourhood_group	0
neighbourhood	0
latitude	0
longitude	0
room_type	1
price	1
minimum_nights	1
number_of_reviews	1
last_review	3701
reviews_per_month	3701
calculated_host_listings_count	1
availability_365	1

dtype: int64

```
missing = df.isnull().sum()

missing_percentage = (missing / len(df)) * 100

missing_summary = pd.DataFrame({
    "Missing Values": missing,
    "Missing Percentage": missing_percentage
})

missing_summary = missing_summary[missing_summary["Missing Values"] > 0]

missing_summary.sort_values(
```

```
    by="Missing Percentage",
    ascending=False
)
```

	Missing Values	Missing Percentage
last_review	3701	16.777732
reviews_per_month	3701	16.777732
name	15	0.067999
host_name	14	0.063466
room_type	1	0.004533
price	1	0.004533
number_of_reviews	1	0.004533
minimum_nights	1	0.004533
calculated_host_listings_count	1	0.004533
availability_365	1	0.004533

```
# Check listings with missing reviews_per_month
df[df["reviews_per_month"].isnull()][
    ["number_of_reviews", "last_review", "reviews_per_month"]
].head(10)
```

	number_of_reviews	last_review	reviews_per_month
2	0.0	NaN	NaN
19	0.0	NaN	NaN
26	0.0	NaN	NaN
36	0.0	NaN	NaN
38	0.0	NaN	NaN
193	0.0	NaN	NaN
204	0.0	NaN	NaN
260	0.0	NaN	NaN
265	0.0	NaN	NaN
267	0.0	NaN	NaN

```
# Check the number of reviews for rows with missing monthly review data
df.loc[
    df["reviews_per_month"].isnull(),
    "number_of_reviews"
].value_counts(dropna=False)
```

number_of_reviews	count
0.0	3700
NaN	1

dtype: int64

```
df[df["number_of_reviews"].isnull()][
    ["name", "number_of_reviews", "last_review", "reviews_per_month", "price"]
]
```

	name	number_of_reviews	last_review	reviews_per_month	price
22058	Great room in a spacious Inwood Apartment	NaN	NaN	NaN	NaN

```
# Check missing prices before cleaning
print("Missing prices before:", df["price"].isnull().sum())

# Remove rows where price is missing
df = df.dropna(subset=["price"])

# Check missing prices after cleaning
print("Missing prices after:", df["price"].isnull().sum())
```

```
Missing prices before: 1
Missing prices after: 0
```

```
# Fill missing monthly review values with 0
df["reviews_per_month"] = df["reviews_per_month"].fillna(0)

# Check the remaining missing values
print("Missing reviews_per_month:", df["reviews_per_month"].isnull().sum())
```

```
Missing reviews_per_month: 0
```

```
# Remove the last_review column
df = df.drop(columns=['last_review'])

# Checking the remaining columns
print(df.columns)
```

```
Index(['id', 'name', 'host_id', 'host_name', 'neighbourhood_group',
       'neighbourhood', 'latitude', 'longitude', 'room_type', 'price',
       'minimum_nights', 'number_of_reviews', 'reviews_per_month',
       'calculated_host_listings_count', 'availability_365'],
      dtype='object')
```

```
# count missing values in each column
print(df.isnull().sum())
```

```
id                                0
name                             15
host_id                           0
host_name                        14
neighbourhood_group               0
neighbourhood                     0
latitude                          0
longitude                         0
room_type                         0
price                             0
minimum_nights                    0
number_of_reviews                 0
reviews_per_month                 0
calculated_host_listings_count    0
availability_365                  0
dtype: int64
```

```
# Remove columns we won't use in our initial model
df = df.drop(columns=["name", "host_name", "id", "host_id"], errors="ignore")

# Display the remaining columns
print(df.columns)

print(df.isnull().sum())
```

```
Index(['neighbourhood_group', 'neighbourhood', 'latitude', 'longitude',
       'room_type', 'price', 'minimum_nights', 'number_of_reviews',
       'reviews_per_month', 'calculated_host_listings_count',
       'availability_365'],
      dtype='object')
neighbourhood_group               0
neighbourhood                     0
```

```
latitude                        0
longitude                       0
room_type                       0
price                           0
minimum_nights                  0
number_of_reviews               0
reviews_per_month               0
calculated_host_listings_count  0
availability_365                0
dtype: int64
```

```
# Check price statistics
print("PRICE STATISTICS")
print(df["price"].describe())

# Check minimum nights statistics
print("\nMINIMUM NIGHTS STATISTICS")
print(df["minimum_nights"].describe())
```

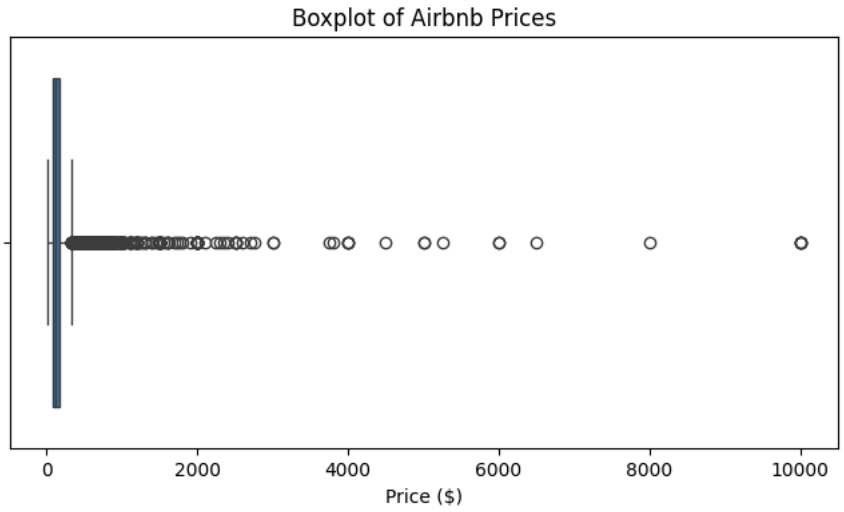
```
PRICE STATISTICS
count    22058.000000
mean       151.245852
std        235.534537
min         10.000000
25%         73.000000
50%        110.000000
75%        175.000000
max      10000.000000
Name: price, dtype: float64

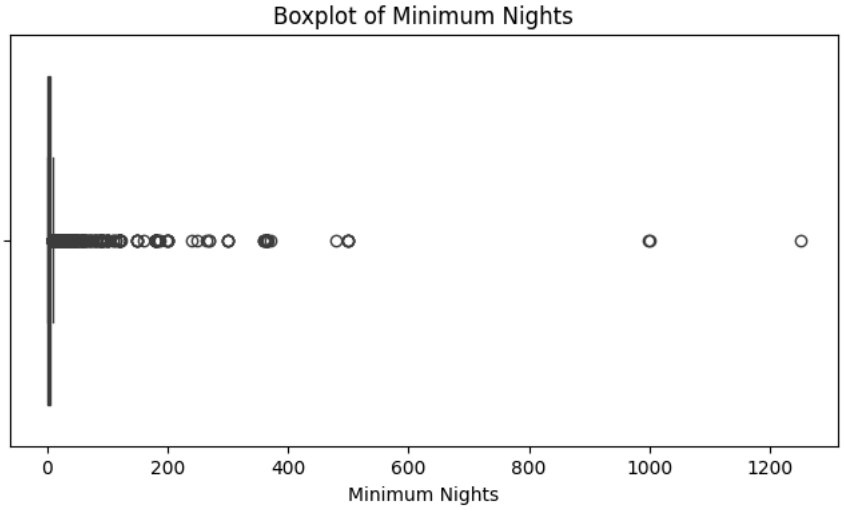
MINIMUM NIGHTS STATISTICS
count    22058.000000
mean         7.408151
std         23.883845
min          1.000000
25%          2.000000
50%          3.000000
75%          5.000000
max       1250.000000
Name: minimum_nights, dtype: float64
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Boxplot for price
plt.figure(figsize=(8, 4))
sns.boxplot(x=df["price"])
plt.title("Boxplot of Airbnb Prices")
plt.xlabel("Price ($)")
plt.show()

# Boxplot for minimum nights
plt.figure(figsize=(8, 4))
sns.boxplot(x=df["minimum_nights"])
plt.title("Boxplot of Minimum Nights")
plt.xlabel("Minimum Nights")
plt.show()
```






```
# Calculate the 95th percentile for price
price_limit = df["price"].quantile(0.95)

# Calculate the 95th percentile for minimum nights
nights_limit = df["minimum_nights"].quantile(0.95)

print("95th percentile price:", price_limit)
print("95th percentile minimum nights:", nights_limit)
```

```
95th percentile price: 350.0
95th percentile minimum nights: 30.0
```

```
# Record the number of rows before removing outliers
print("Rows before cleaning:", len(df))

# Keep listings at or below the 95th-percentile price
df = df[df["price"] <= price_limit]

# Remove listings requiring 365 nights or more
df = df[df["minimum_nights"] < 365]

# Check the number of rows after cleaning
print("Rows after cleaning:", len(df))

# Verify the maximum values
print("Maximum price:", df["price"].max())
print("Maximum minimum nights:", df["minimum_nights"].max())
```

```
Rows before cleaning: 22058
Rows after cleaning: 21053
Maximum price: 350.0
Maximum minimum nights: 364.0
```

```
# Count unique values in neighborhood columns
print("Unique neighborhoods:", df["neighbourhood"].nunique())

print(
    "Unique neighborhood groups:",
    df["neighbourhood_group"].nunique()
)

print("Unique room types:", df["room_type"].nunique())
```

```
Unique neighborhoods: 203
Unique neighborhood groups: 5
Unique room types: 3
```

```
df = df.drop(columns=["neighbourhood"])

print(df.columns)
```

```
Index(['neighbourhood_group', 'latitude', 'longitude', 'room_type', 'price',
       'minimum_nights', 'number_of_reviews', 'reviews_per_month',
       'calculated_host_listings_count', 'availability_365'],
      dtype='object')
```

```
# Convert categorical columns into numerical columns
df = pd.get_dummies(
    df,
    columns=["neighbourhood_group", "room_type"],
    drop_first=True,
    dtype=int
)

# Display the updated columns
print(df.columns)
```

```
Index(['latitude', 'longitude', 'price', 'minimum_nights', 'number_of_reviews',
       'reviews_per_month', 'calculated_host_listings_count',
       'availability_365', 'neighbourhood_group_Brooklyn',
       'neighbourhood_group_Manhattan', 'neighbourhood_group_Queens',
       'neighbourhood_group_Staten Island', 'room_type_Private room',
       'room_type_Shared room'],
      dtype='object')
```

```
# Features: everything except price
X = df.drop(columns=["price"])

# Target: the price we want to predict
y = df["price"]

# Check their shapes
print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (21053, 13)
Target shape: (21053,)
```

```
from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# Display the shapes
print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

```
X_train: (16842, 13)
X_test: (4211, 13)
y_train: (16842,)
y_test: (4211,)
```

```
from sklearn.linear_model import LinearRegression

# Create the model
model = LinearRegression()

# Train the model using the training data
model.fit(X_train, y_train)

print("Linear Regression model trained successful")
```

```
Linear Regression model trained successful
```

```
# Generate predictions on the test set
y_pred = model.predict(X_test)

# Compare actual prices with predicted prices
comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

print(comparison.head(10).round(2))
```

```
   Actual Price  Predicted Price
0         200.0           180.93
1          65.0            74.86
2         170.0           181.37
3         240.0           183.17
4          75.0           139.65
5          95.0            83.17
6         315.0           183.70
7          58.0           103.80
8         160.0           153.80
9          87.0           108.08
```

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

# Calculate evaluation metrics
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

# Display the results
print("R2 Score:", round(r2, 4))
print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
```

```
R2 Score: 0.4754
MAE: 36.53
RMSE: 49.45
```

```
import matplotlib.pyplot as plt

# Plot actual prices against predicted prices
plt.figure(figsize=(8, 6))

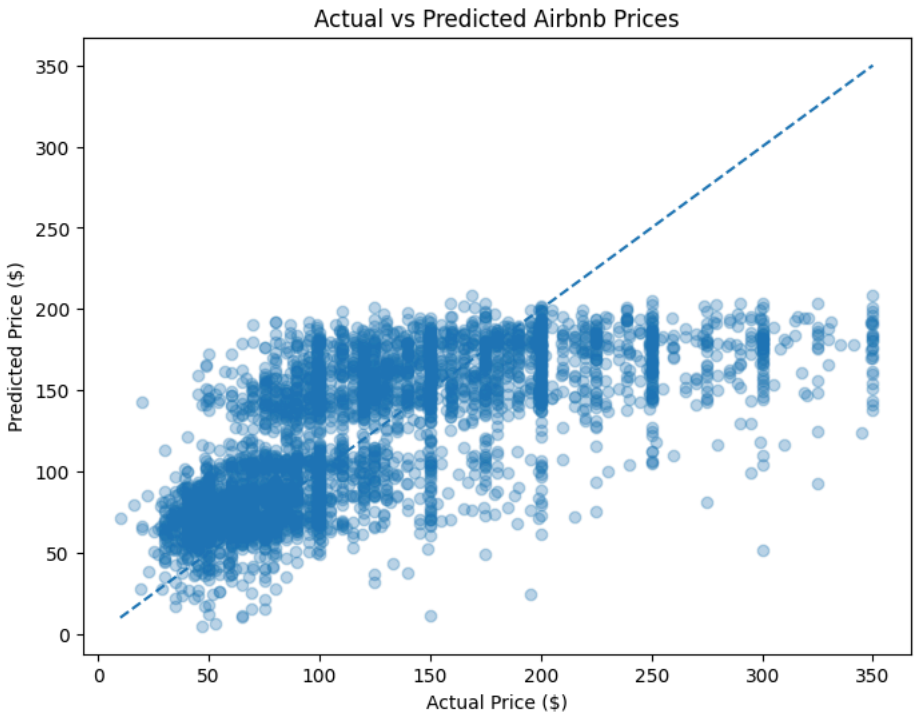
plt.scatter(y_test, y_pred, alpha=0.3)

# Ideal prediction line
```

```
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--"
)

plt.xlabel("Actual Price ($)")
plt.ylabel("Predicted Price ($)")
plt.title("Actual vs Predicted Airbnb Prices")

plt.show()
```

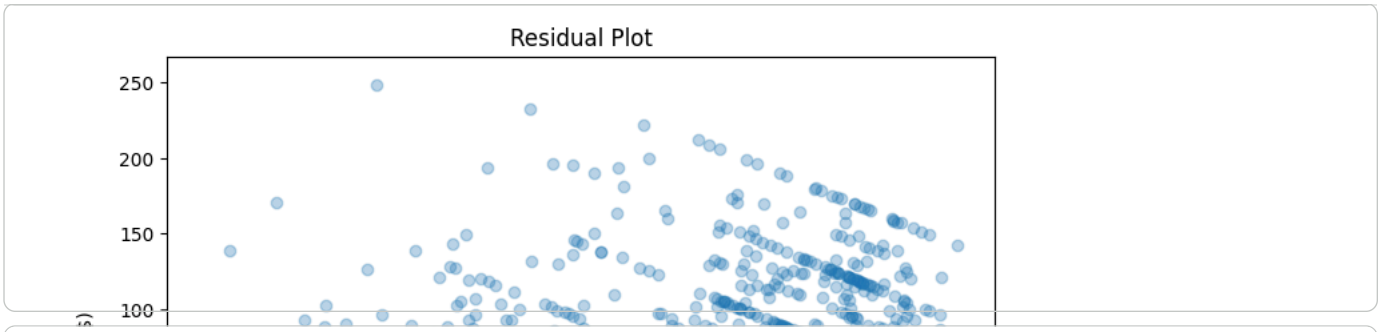



```
# Calculate residuals
residuals = y_test - y_pred

# Plot residuals against predicted prices
plt.figure(figsize=(8, 6))
plt.scatter(y_pred, residuals, alpha=0.3)

# Reference line at zero
plt.axhline(y=0, linestyle="--")

plt.xlabel("Predicted Price ($)")
plt.ylabel("Residual ($)")
plt.title("Residual Plot")
plt.show()
```




```
plt.figure(figsize=(8, 5))

plt.hist(residuals, bins=40)

plt.xlabel("Residual ($)")
plt.ylabel("Number of Listings")
plt.title("Distribution of Prediction Errors")

plt.show()
```

